# Project Report

# Placement Portal Application

## Student Details


- **Name:** Snehapriya Thiagarajan
- **University:** Indian Institute of Science Education and Research Bhopal
- **Degree:** BS in Electronics and Communication Engineering
- **Registration Number:** 23f3001500


## Problem Statement

The main goal of this project was to develop a unified digital Placement Portal to centralize campus recruitment. Typically, placement activities are scattered, so this platform serves as a bridge. It gives companies a dedicated space to announce job openings and evaluate candidates, while students get an interface to browse opportunities and submit their applications. Administrators are also provided with the necessary tools to monitor activity and ensure the entire recruitment runs smoothly .

## Our Approach

I approached this project by building a web-based application that connects students, companies, and the college admin in one place. I used Flask for the backend and a simple Vue.js setup for the frontend. First, I created different dashboards so that Admins can approve Companies and placement drives, Companies can post placement drives, and Students can view the available jobs and apply for them. To make the system work better, I added backend rules that automatically check a student's CGPA before allowing them to apply for a job. I also integrated a Celery asynchronous batch job that allows students to export their complete placement history as a CSV file, which sends an alert upon completion. Finally, I used background tools like Celery to automatically send update emails, deadline reminders, and offer letters to students without making the website slow or frozen.

## AI/LLM Declaration

AI tools were used in a limited and supportive manner during the development of this project. They were mainly used for debugging, small implementation guidance, and assistance while designing parts of the frontend user interface.However, I independently designed and verified the core system architecture, database schema, and the actual application logic. Overall, AI assistance accounts for roughly 20-25 % of the entire development cycle.

## Frameworks and Libraries Used

- Flask – The core backend framework used to build the REST API
- Vue.js – Frontend framework for creating the Single Page Application
- Vue Router – Takes care of the frontend routing and navigation without page reloads
- Flask SQLAlchemy – The ORM and local database used to manage and store all application data.

- SQLite - database for storing application data.
- Jinja2 -The templating engine utilized to render the base HTML views.
- Bootstrap 5 -CSS framework used for good design and layout
- Flask JWT Extended – Provides secure JWT-based authentication and role-based access control
- Axios – Used for sending asynchronous API requests from frontend to backend
- Celery – Task queue used to run background jobs asynchronously
- Redis – Message broker and caching system used with Celery
- Flask Caching – Library used to implement server side caching

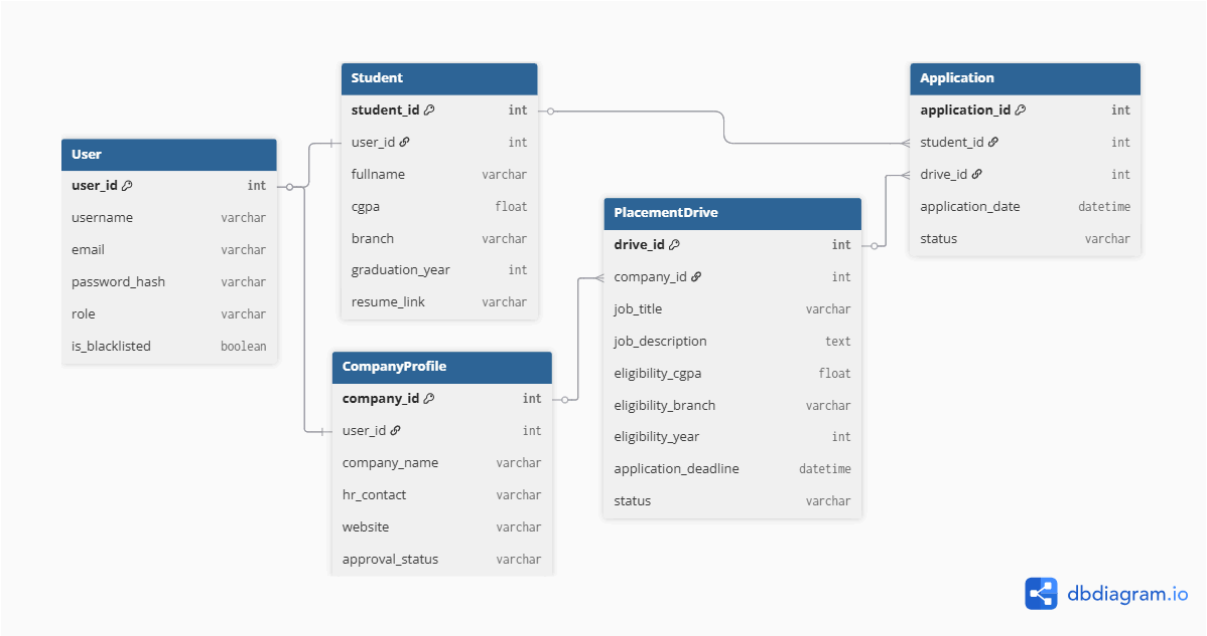
## Database Schema / ER Diagram

### Tables:

- **User** - stores core authentication details (user_id, username, email, password_hash, role, is_blacklisted)
- **Student** - stores student details (student_id, user_id, fullname, cgpa, branch, graduation_year, resume_link)
- **CompanyProfile** - stores registered company details (company_id, user_id, company_name, hr_contact, website, approval_status)
- **PlacementDrive** - stores job drives created by companies (drive_id, company_id, job_title, job_description, eligibility_cgpa, eligibility_branch, eligibility_year, application_deadline, status)
- **Application** - saves student application history (application_id, student_id, drive_id, application_date, status)

### Relationships:

- One-to-One → User → Student
- One-to-One → User → CompanyProfile
- One-to-Many → CompanyProfile → PlacementDrive
- One-to-Many → Student → Application
- One-to-Many → PlacementDrive → Application




## API Resource Endpoints

I created RESTful API endpoints for the four main elements of the system: Authentication, Admin actions , Company actions, and Student actions.

- Authentication: Endpoints like "/api/login" and "/api/register" were implemented to check users and make JWT tokens.
- Admin: Endpoints were created to get system statistics and manage approvals .
- Companies & Students: Endpoints were implemented to handle obtaining job lists (GET requests) and submitting data like new job postings or job applications (POST requests).

All protected APIs were implemented using the @jwt_required() decorator to ensure only logged-in users with the correct roles can use them. Data is sent back and forth between the Vue.js frontend and Flask backend in JSON format.

## Architecture and Features

### Architecture Overview:

- app.py – The main backend file that runs the Flask application, defines the database models using SQLAlchemy, handles background tasks with Celery, and contains all the API routes.
- templates/index.html – The main frontend file where the Single Page Application starts.
- static/app.js- This contains the JS files.
- placement_portal.db – A simple SQLite database file automatically created by SQLAlchemy to store the application data.
- Redis & Celery – Used to manage background tasks and improve performance by handling caching and message passing.

### Implemented Features:

- Entry based on user type and login system for different users such as Admin, Students, and Company HR.
- Entry checking  using password hashing and JSON Web Tokens  to manage user sessions.
- User interface built with Bootstrap 5 that adapts smoothly to both desktop and mobile devices.
- Real-time, eligibility-based search and filtering on the Student Dashboard using Vue.js
- Backend validation rules to ensure conditions such as minimum CGPA are checked before allowing students to apply for jobs.
- User-triggered asynchronous batch jobs allowing students to export placement application history into CSV formats.
- Automated scheduled background jobs via Celery Beat to send daily deadline reminders to students and monthly HTML placement activity reports to the Admin.

Video link

MAD2 PLACEMENT PORTAL 23F3001500